

Case Study 1 — Bayesian Linear Regression: Gradient-Based Optimisation and Posterior Variance Collapse

David Ewing (82171165)

5 May 2026

Abstract

Mean-field Variational Inference (VI) provides a tractable approximation to Bayesian posterior computation, but systematically underestimates posterior uncertainty — a failure known as *posterior variance collapse*. This report investigates variance collapse in Bayesian linear regression using the gradient-based optimisation methods taught in ENEL445. Variational inference recasts posterior computation as maximisation of the Evidence Lower Bound (ELBO), making it a natural testbed for gradient ascent, Newton’s method, and BFGS. Coordinate Ascent Variational Inference (CAVI) provides the analytical baseline, exploiting the conjugate Normal–Gamma structure of the model.

The ELBO is derived in closed form under a mean-field factorisation. The variational parameters are $\boldsymbol{\mu}_\beta$, $\boldsymbol{\Sigma}_\beta$, a_e , and b_e ; Cholesky and log-space reparameterisations keep the covariance matrix positive definite and the Gamma parameters positive throughout optimisation. For each gradient-based method, the report states the search direction, step-size rule, and exit conditions. Line-search methods require the ascent condition

$$\nabla \text{ELBO}(\boldsymbol{\phi}^{(t)})^T \boldsymbol{d}^{(t)} > 0;$$

gradient ascent uses Armijo backtracking and BFGS uses Wolfe conditions to maintain the curvature condition $\boldsymbol{y}_t^T \boldsymbol{s}_t > 0$ for the inverse-Hessian update.

The methods are compared on ELBO convergence, computational cost, posterior mean accuracy, and posterior standard-deviation ratios against a Gibbs sampler reference. A secondary comparison uses an entropy-regularised ELBO to examine whether explicitly incentivising broader posteriors can reduce collapse.

Introduction

Bayesian linear regression is the simplest non-trivial model in which all components of variational inference—the ELBO, its gradient, and the posterior approximation—can be derived and verified in closed form, making it the natural baseline for comparing optimisation methods.

Bayesian inference provides principled uncertainty quantification but requires computing intractable posterior distributions. For most real-world models, the posterior integral,

$$p(\boldsymbol{\theta} \mid \mathbf{y}) = \frac{p(\mathbf{y} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta})}{\int p(\mathbf{y} \mid \boldsymbol{\theta}) p(\boldsymbol{\theta}) d\boldsymbol{\theta}},$$

is typically intractable. In this case study, $\boldsymbol{\theta} = (\boldsymbol{\beta}, \tau_e)$ —the regression coefficients and the observation noise precision—so the posterior of interest is

$$p(\boldsymbol{\beta}, \tau_e \mid \mathbf{y}) \propto p(\mathbf{y} \mid \boldsymbol{\beta}, \tau_e) p(\boldsymbol{\beta}) p(\tau_e).$$

The substitution $\boldsymbol{\theta} = (\boldsymbol{\beta}, \tau_e)$ is specific to this case study; other case studies in this project define $\boldsymbol{\theta}$ according to their own model parameters. Conjugate Gaussian–Gamma priors make this case study a deliberate exception: the posterior is analytically tractable and CAVI coordinate updates are closed-form, providing a verified reference against which gradient-based VI methods can be compared. Variational Inference (VI) converts this inference problem into an optimisation problem, making it directly amenable to the gradient-based methods in this course.

Instead of computing the true posterior, VI finds the “best” approximation $q(\boldsymbol{\theta})$ from a tractable family $Q(\boldsymbol{\theta})$ by maximising the Evidence Lower Bound (ELBO). This transforms intractable integration into tractable optimisation—a natural testbed for the gradient-based methods.

A well-known failure mode of mean-field VI is *posterior variance collapse*: the optimised approximation $q(\boldsymbol{\theta})$ is systematically overconfident, underestimating posterior uncertainty. This is a direct consequence of the KL direction minimised during ELBO maximisation, compounded by the mean-field factorisation assumption.¹

This paper applies four optimisation methods—CAVI, gradient ascent, Newton’s method, and BFGS—to the ELBO and compares their convergence behaviour, computational cost, and posterior quality. As a secondary investigation, the ELBO is augmented with an entropy regularisation term $\lambda H[q(\boldsymbol{\beta})]$,

$$\mathcal{L}_{\text{reg}}(\phi) = \mathcal{L}(\phi) + \lambda H[q(\boldsymbol{\beta})],$$

to examine whether explicitly incentivising broader posteriors can mitigate variance collapse.

Throughout all gradient-based methods, the variational covariance $\boldsymbol{\Sigma}_{\boldsymbol{\beta}}$ must remain symmetric positive definite at every iteration.²

The classical foundation for the variational framework used here is least-squares regression, as developed in lectures by Prof. Roy S. Smith (Erskine Visiting Fellow, ETH Zürich, ENEL445 2026). The design matrix $\mathbf{X}$ ³ and coefficient vector $\boldsymbol{\beta}$ ⁴ follow the standard over-determined formulation ($n > p$ ⁵), so that the normal equations $(\mathbf{X}^T \mathbf{X})\boldsymbol{\beta} = \mathbf{X}^T \mathbf{y}$ have a unique solution and the Bayesian posterior mean reduces to this solution in the limit of a flat prior. The noise model adopted here—errors on $\mathbf{y}$ only, with noise precision τ_e —corresponds to the equation-error form; this is an explicit modelling assumption rather than a derived result.

¹See Appendix [Appendix A](#) for the full mechanistic explanation.

²This structural constraint is satisfied automatically via Cholesky reparameterisation; see [Appendix B](#).

³Denoted Φ and called the *regressor matrix* in lectures (Prof. R. S. Smith, ENEL445 2026).

⁴Denoted $\boldsymbol{\theta}$ in lectures.

⁵Denoted d in lectures.

0.1 Numerical Study and Required Optimisation Components

This report formulates variational inference for Bayesian linear regression as a constrained optimisation problem and applies four methods: Coordinate Ascent Variational Inference (CAVI), gradient ascent, Newton's method, and BFGS. Each method is characterised by four components from the course framework: entry conditions, search direction, line search, and exit conditions. All four methods share the same initial point so that comparisons reflect method behaviour rather than initialisation.

Entry Conditions

All methods are initialised at the prior mean: $\boldsymbol{\mu}_\beta = \mathbf{0}$, $\boldsymbol{\Sigma}_\beta = \mathbf{I}_p$, $a_e = \alpha_e$, $b_e = \gamma_e$. In the unconstrained parameterisation this corresponds to $\mathbf{L} = \mathbf{I}_p$ (so $\tilde{\ell}_{ii} = 0$ and off-diagonal entries zero), $\eta_a = \log \alpha_e$, and $\eta_b = \log \gamma_e$. CAVI does not require an explicit initialisation of the search direction and uses the same starting variational parameters.

Search Direction

For all gradient-based methods, the proposed search direction must be an ascent direction before a step is accepted:

$$\nabla \text{ELBO}(\boldsymbol{\phi}^{(t)})^T \mathbf{d}^{(t)} > 0. \quad (1)$$

Gradient ascent uses $\mathbf{d}^{(t)} = \nabla \mathcal{L}(\boldsymbol{\xi}^{(t)})$ directly. Newton's method uses $\mathbf{d}^{(t)} = -[\nabla^2 \mathcal{L}(\boldsymbol{\xi}^{(t)})]^{-1} \nabla \mathcal{L}(\boldsymbol{\xi}^{(t)})$, falling back to the gradient direction if the Hessian is not positive definite. BFGS approximates the inverse Hessian as $\mathbf{B}_t$ and uses $\mathbf{d}^{(t)} = \mathbf{B}_t \nabla \mathcal{L}(\boldsymbol{\xi}^{(t)})$. CAVI replaces the search direction with closed-form coordinate updates derived analytically from the model.

Line Search

The step size $\alpha^{(t)}$ is determined per method. Gradient ascent uses Armijo backtracking: starting from an initial step size, $\alpha^{(t)}$ is halved until sufficient increase in the ELBO is achieved. BFGS uses Wolfe conditions, combining sufficient increase with a curvature condition that ensures

$$\mathbf{y}_t^T \mathbf{s}_t > 0, \quad (2)$$

where $\mathbf{s}_t = \boldsymbol{\xi}^{(t+1)} - \boldsymbol{\xi}^{(t)}$ and $\mathbf{y}_t = \nabla \mathcal{L}(\boldsymbol{\xi}^{(t+1)}) - \nabla \mathcal{L}(\boldsymbol{\xi}^{(t)})$. This positivity condition is required for the BFGS update to preserve positive definiteness of $\mathbf{B}_t$. Newton's method uses a fixed step size of 1 unless the Hessian is indefinite. CAVI has no line search: each coordinate update exactly maximises the ELBO with respect to that parameter in closed form.

Exit Conditions

Exit conditions are applied uniformly across the optimisation methods. Iteration terminates when any of the following conditions is satisfied:

- (i) Gradient norm: $\|\nabla \text{ELBO}\| < \varepsilon_g$.
- (ii) Relative ELBO change: $|\Delta \text{ELBO}|/|\text{ELBO}| < \varepsilon_r$.
- (iii) Parameter change: $\|\Delta \boldsymbol{\phi}\| < \varepsilon_\phi$.
- (iv) Maximum iterations: $t \geq t_{\max}$.

CAVI additionally monitors the maximum relative change in its variational parameters:

$$\max_i \frac{|\phi_i^{(t+1)} - \phi_i^{(t)}|}{|\phi_i^{(t+1)}|} < \varepsilon. \quad (3)$$

0.2 Bayesian Regression Model

The Bayesian regression model is

$$\mathbf{y} \sim \mathcal{N}(\mathbf{X}\boldsymbol{\beta}, \tau_e^{-1}\mathbf{I}_n) \quad (\text{likelihood}), \quad (4)$$

$$\boldsymbol{\beta} \sim \mathcal{N}(\mathbf{0}, \lambda_\beta^{-1}\mathbf{I}_p) \quad (\text{prior on coefficients}), \quad (5)$$

$$\tau_e \sim \text{Gamma}(\alpha_e, \gamma_e) \quad (\text{prior on precision}). \quad (6)$$

Here, $\mathbf{X}$ is the design matrix, $\mathbf{y}$ is the response vector, $\boldsymbol{\beta}$ is the coefficient vector, and τ_e is the observation-noise precision.

0.3 Mean-Field Variational Approximation

The mean-field approximation is

$$q(\boldsymbol{\beta}, \tau_e) = q(\boldsymbol{\beta}) q(\tau_e), \quad (7)$$

$$q(\boldsymbol{\beta}) = \mathcal{N}(\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta), \quad (8)$$

$$q(\tau_e) = \text{Gamma}(a_e, b_e). \quad (9)$$

The variational parameters are

$$\boldsymbol{\phi} = (\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta, a_e, b_e), \quad (10)$$

with dimension $d = p + p(p+1)/2 + 2$. For $p = 3$, this gives $d = 3 + 6 + 2 = 11$ parameters to optimise.

0.4 Key Notation

Symbol	Meaning
Bayesian model	
$\mathbf{X}$	Design matrix ($n \times p$)
$\mathbf{y}$	Response vector
$\boldsymbol{\beta}$	Regression coefficient vector
τ_e	Observation-noise precision
α_e, γ_e	Shape and rate of the prior on τ_e
p	Number of predictors
n	Number of observations
Variational approximation	
$q(\cdot)$	Variational approximating distribution
$\boldsymbol{\phi}$	Full variational parameter vector $(\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta, a_e, b_e)$
$\boldsymbol{\mu}_\beta$	Mean of variational posterior $q(\boldsymbol{\beta})$
$\boldsymbol{\Sigma}_\beta$	Covariance of variational posterior $q(\boldsymbol{\beta})$
a_e, b_e	Shape and rate of variational $q(\tau_e) = \text{Gamma}(a_e, b_e)$
$\mathcal{L}(\boldsymbol{\phi})$	Evidence Lower Bound (ELBO)
$H[q(\boldsymbol{\beta})]$	Differential entropy of $q(\boldsymbol{\beta})$
λ	Entropy regularisation weight
Reparameterisation	
$\mathbf{L}$	Lower-triangular Cholesky factor, $\boldsymbol{\Sigma}_\beta = \mathbf{L}\mathbf{L}^T$
$\tilde{\ell}_{ii}$	Log-diagonal entry, $L_{ii} = e^{\tilde{\ell}_{ii}}$ (ensures $L_{ii} > 0$)
$\boldsymbol{\xi}$	Unconstrained optimisation vector $(\boldsymbol{\mu}_\beta, \tilde{\ell}_{11}, \ell_{21}, \dots, \eta_a, \eta_b)$
η_a, η_b	Log-space parameters, $a_e = e^{\eta_a}$, $b_e = e^{\eta_b}$
Optimisation	
$\mathbf{d}^{(t)}$	Search direction at iteration t
α_t	Step size (line-search parameter) at iteration t
$\nabla \mathcal{L}$	Gradient of the ELBO w.r.t. $\boldsymbol{\xi}$
$\nabla^2 \mathcal{L}$	Hessian of the ELBO w.r.t. $\boldsymbol{\xi}$
$\mathbf{B}_t$	BFGS inverse-Hessian approximation at iteration t
Special functions	
$\psi(x)$	Digamma function, $\psi(x) = \frac{d}{dx} \log \Gamma(x)$
$\psi^{(1)}(x)$	Trigamma function, $\psi^{(1)}(x) = \frac{d}{dx} \psi(x)$
Lecture correspondence (Prof. R. S. Smith, ENEL445 2026)	
Φ	Regressor matrix = $\mathbf{X}$ in this report
θ	Coefficient vector = $\boldsymbol{\beta}$ in this report
d	Number of parameters = p in this report
$J(\theta)$	Cost function = $\mathcal{L}(\boldsymbol{\phi})$ (ELBO) in this report

1 Optimisation Problem Formulation

All four methods—CAVI, gradient ascent, Newton’s method, and BFGS—operate on the same objective and respect the same constraints. The formulation below follows the standard form from the course, so that differences in performance reflect the methods themselves rather than problem differences.

Decision Variables

For Bayesian linear regression with p predictors, the variational parameters (decision variables) are:

- $\boldsymbol{\mu}_\beta \in \mathbb{R}^p$: posterior mean of regression coefficients
- $\boldsymbol{\Sigma}_\beta \in \mathbb{R}^{p \times p}$: posterior covariance (symmetric positive definite, $\boldsymbol{\Sigma}_\beta \succ 0$)
- $a_e, b_e > 0$: shape and rate parameters of the variational Gamma factor for precision

Here, a_e and b_e parameterise $q(\tau_e) = \text{Gamma}(a_e, b_e)$; the model prior on τ_e is specified in the Test Case section.

Total dimension: $d = p + \frac{p(p+1)}{2} + 2 = 11$ variables for $p = 3$ predictors.

Objective Function

Maximise the Evidence Lower Bound (ELBO):

$$\text{ELBO}(\phi) = \mathbb{E}_q[\log p(\mathbf{y}, \boldsymbol{\beta}, \tau_e)] + H(q) \quad (11)$$

$$= \mathbb{E}_q[\log p(\mathbf{y}, \boldsymbol{\beta}, \tau_e)] - \mathbb{E}_q[\log q(\boldsymbol{\beta}, \tau_e)]. \quad (12)$$

where $\phi = (\boldsymbol{\mu}_\beta, \boldsymbol{\Sigma}_\beta, a_e, b_e)$ are the variational parameters. The first term, $\mathbb{E}_q[\log p(\mathbf{y}, \boldsymbol{\beta}, \tau_e)]$, is the expected log joint: it rewards approximations q that place mass where the data and prior are jointly well explained. The second term, $H(q) = -\mathbb{E}_q[\log q(\boldsymbol{\beta}, \tau_e)]$, is the *differential entropy* of the approximating distribution: it rewards approximations that are broad and diffuse, penalising overconfident solutions. Maximising the ELBO therefore balances data fit against posterior spread — the entropy term is precisely what variance collapse works against.

Constraints

- $\boldsymbol{\Sigma}_\beta \succ 0$: Covariance must be strictly positive definite; $\boldsymbol{\Sigma}_\beta^{-1}$ appears in the ELBO and CAVI updates, so a semi-definite covariance causes those computations to fail.
- $a_e, b_e > 0$: Gamma parameters must be positive
- Optional: Minimum variance constraints $\sigma_{\beta_j}^2 \geq \sigma_{\min}^2$ to prevent underdispersion

Constraint Handling via Reparameterisation:

To enforce $\boldsymbol{\Sigma}_\beta \succ 0$, use Cholesky decomposition:

$$\boldsymbol{\Sigma}_\beta = \mathbf{L}\mathbf{L}^T \quad (13)$$

where $\mathbf{L} \in \mathbb{R}^{p \times p}$ is lower triangular with positive diagonal entries. The unconstrained parameters are:

- $L_{ii} = e^{\ell_{ii}}$ for $i = 1, \dots, p$ (exponentiate to ensure positivity)
- $L_{ij} = \ell_{ij}$ for $i > j$ (off-diagonal entries unconstrained)

Total Cholesky parameters: $\frac{p(p+1)}{2}$ elements of $\boldsymbol{\ell}$.

To enforce $a_e, b_e > 0$, use log-space parameterisation:

$$a_e = e^{\eta_a}, \quad b_e = e^{\eta_b} \quad (14)$$

where $\eta_a, \eta_b \in \mathbb{R}$ are unconstrained.

Unconstrained parameter vector: $\boldsymbol{\xi} = (\boldsymbol{\mu}_\beta, \boldsymbol{\ell}, \eta_a, \eta_b) \in \mathbb{R}^d$ with $d = p + \frac{p(p+1)}{2} + 2 = 11$ for $p = 3$.

2 Posterior Variance Collapse

Because this case study uses conjugate Gaussian–Gamma priors, the true posterior $p(\boldsymbol{\beta}, \tau_e \mid \mathbf{y})$ is analytically available as a Normal–Gamma distribution. This makes this case study the only one in this project where posterior variance collapse can be measured against ground truth, not merely against a sampler reference. The Gibbs sampler still serves as a practical reference, but the true posterior provides an exact benchmark against which both the VI approximation and the sampler can be validated.

Posterior variance collapse refers to the tendency of mean-field VI to produce posterior approximations with variances that are too small. In Bayesian regression, this appears as a covariance matrix $\boldsymbol{\Sigma}_\beta$ whose diagonal entries underestimate the uncertainty in the regression coefficients. The resulting posterior intervals are too narrow, making the model appear more confident than it should be.

Convergence speed and ELBO value are necessary measures of optimiser performance, but they are not sufficient. An optimisation method may converge reliably and achieve a high ELBO while still producing a posterior approximation that understates uncertainty—and in Bayesian inference, calibrated uncertainty is a primary concern, not an afterthought. Both convergence quality and posterior dispersion are evaluated in this report.

2.1 Why Collapse Occurs

Mean-field VI restricts the approximating family by assuming

$$q(\boldsymbol{\beta}, \tau_e) = q(\boldsymbol{\beta}) q(\tau_e), \quad (15)$$

which forces independence between the regression coefficients and the noise precision. In this case study the true Normal–Gamma posterior contains genuine dependence between $\boldsymbol{\beta}$ and τ_e : a larger noise precision implies tighter coefficient estimates, so the two are correlated in the exact posterior. The mean-field factorisation removes this dependence by assumption, discarding uncertainty that would otherwise appear through the joint structure.

The second cause is the direction of the KL divergence implied by the ELBO. Standard VI minimises

$$\begin{aligned} \text{KL}[q\|p] &= \int q(\boldsymbol{\theta}) \log \frac{q(\boldsymbol{\theta})}{p(\boldsymbol{\theta} \mid \mathbf{y})} d\boldsymbol{\theta} \\ &= \int_0^\infty \int_{\mathbb{R}^p} q(\boldsymbol{\beta}, \tau_e) \log \frac{q(\boldsymbol{\beta}, \tau_e)}{p(\boldsymbol{\beta}, \tau_e \mid \mathbf{y})} d\boldsymbol{\beta} d\tau_e \\ &= \int_0^\infty \int_{\mathbb{R}^p} q(\boldsymbol{\beta}) q(\tau_e) \log \frac{q(\boldsymbol{\beta}) q(\tau_e)}{p(\boldsymbol{\beta}, \tau_e \mid \mathbf{y})} d\boldsymbol{\beta} d\tau_e. \end{aligned} \quad (16)$$

This objective strongly penalises q for placing mass where the true posterior is small, but does not penalise q for failing to cover regions where the true posterior is large. The result is that q locks onto the high-probability core of $p(\boldsymbol{\beta}, \tau_e \mid \mathbf{y})$ and ignores the tails, systematically underestimating posterior variance. Because the true posterior is available analytically in this case study, this effect can be quantified exactly rather than inferred from samples.

2.2 Diagnostic

The numerical experiments compare the variational posterior against two references: the analytically derived true posterior (available because of conjugate priors) and a Gibbs sampler. For each coefficient β_j , two standard-deviation ratios are computed:

$$\frac{\sigma_{\text{VB}}}{\sigma_{\text{true}}}, \quad \frac{\sigma_{\text{VB}}}{\sigma_{\text{Gibbs}}}. \quad (17)$$

Ratios below one confirm that the variational approximation is narrower than the reference (*posterior variance collapse*). The ratio against the true posterior is the definitive diagnostic; the ratio against Gibbs serves as a consistency check and provides the comparison metric used in the other case studies where the true posterior is not available.

2.3 Approaches Considered

Three approaches are considered for reducing or diagnosing posterior variance collapse. Of these, only entropy regularisation (Approach 3) is implemented in this report; the first two are discussed to motivate that choice.

2.3.1 Informative Prior on τ_e

A stronger prior on τ_e can limit how large the expected precision becomes. Since larger precision corresponds to smaller observation-noise variance, controlling τ_e can indirectly prevent Σ_β from becoming too small. The limitation is that this changes the statistical model and requires a defensible prior choice.

2.3.2 Minimum Variance Constraint

A direct constraint can be imposed on the diagonal entries of Σ_β :

$$\Sigma_{\beta,jj} \geq \sigma_{\min}^2, \quad j = 1, \dots, p. \quad (18)$$

This prevents individual coefficient variances from falling below a chosen threshold. The limitation is that the threshold is external to the original ELBO objective.

2.3.3 Entropy Regularisation

The most direct objective-based approach considered here is entropy regularisation:

$$\mathcal{L}_{\text{reg}}(\phi) = \mathcal{L}(\phi) + \lambda H[q(\beta)]. \quad (19)$$

For $q(\beta) = \mathcal{N}(\mu_\beta, \Sigma_\beta)$, the entropy is

$$H[q(\beta)] = \frac{1}{2} \log \det(2\pi e \Sigma_\beta) \quad (20)$$

$$= \frac{p}{2}(1 + \log 2\pi) + \frac{1}{2} \log \det(\Sigma_\beta). \quad (21)$$

The regularised gradient with respect to the covariance matrix gains the additional term

$$\nabla_{\Sigma_\beta} \mathcal{L}_{\text{reg}} = \nabla_{\Sigma_\beta} \mathcal{L} + \frac{\lambda}{2} \Sigma_\beta^{-1}. \quad (22)$$

This term rewards broader covariance estimates and directly counters the tendency toward overly narrow posterior approximations.

3 ELBO Evaluation

Before running any optimiser, the ELBO and its analytical gradient are verified numerically. This ensures that the objective function being maximised is correctly implemented and that all reparameterisations are consistent.

3.1 Gradient Verification

The analytical gradient $\nabla_{\xi}\mathcal{L}$ is computed with respect to all unconstrained parameters $\xi = (\mu_{\beta}, \tilde{\ell}_{11}, \ell_{21}, \tilde{\ell}_{22}, \eta_a, \eta_b)$. Each component is checked against a central finite-difference approximation:

$$\frac{\partial \mathcal{L}}{\partial \xi_i} \approx \frac{\mathcal{L}(\xi + h\mathbf{e}_i) - \mathcal{L}(\xi - h\mathbf{e}_i)}{2h}, \quad h = 10^{-5}. \quad (23)$$

The relative error between the analytical and numerical gradient must satisfy $\|\nabla_{\text{analytical}} - \nabla_{\text{FD}}\|_{\infty} / \|\nabla_{\text{analytical}}\|_{\infty} \leq 10^{-5}$ for all components before any optimiser is run. The result is displayed as a bar chart (`linear_elbo_gradient_check.png`) comparing analytical and finite-difference values side by side for each unconstrained parameter. The gradient check table is saved to `results/tables/linear_gradient_check.tex`.

3.2 ELBO Landscape

A two-dimensional slice of the ELBO surface is computed by varying μ_{β_0} and μ_{β_1} over a grid while holding all other parameters fixed at their CAVI optimum. This visualises the objective surface that the gradient-based optimisers are navigating and confirms that the CAVI solution lies at the maximum. The result is saved as `linear_elbo_landscape.png`.

4 Optimisation Methods

The optimisation methods are used to investigate posterior variance collapse, not merely to solve the ELBO maximisation problem. Each method is assessed both as a numerical optimiser and as a mechanism for producing a variational posterior whose uncertainty can be compared against a Gibbs sampler reference.

4.1 CAVI Baseline

Coordinate Ascent Variational Inference (CAVI) is the analytical baseline for this conjugate Bayesian regression model. It updates each variational parameter block sequentially using closed-form solutions:

$$\Sigma_{\beta} \leftarrow \left(\frac{a_e}{b_e} \mathbf{X}^T \mathbf{X} + \lambda_{\beta} \mathbf{I}_p \right)^{-1}, \quad (24)$$

$$\mu_{\beta} \leftarrow \frac{a_e}{b_e} \Sigma_{\beta} \mathbf{X}^T \mathbf{y}, \quad (25)$$

$$a_e \leftarrow \alpha_e + \frac{n}{2}, \quad (26)$$

$$b_e \leftarrow \gamma_e + \frac{1}{2} [\|\mathbf{y} - \mathbf{X}\mu_{\beta}\|^2 + \text{tr}(\mathbf{X}^T \mathbf{X} \Sigma_{\beta})]. \quad (27)$$

CAVI has monotonic ELBO increase⁶, linear convergence, and $O(np^2)$ cost per iteration. It provides the benchmark against which the gradient-based methods are checked.

4.2 Gradient Ascent

Gradient ascent applies the first-order update in the unconstrained parameterisation:

$$\xi^{(t+1)} = \xi^{(t)} + \alpha_t \nabla \text{ELBO}(\xi^{(t)}). \quad (28)$$

⁶Monotonic ELBO increase means the objective is non-decreasing across iterations: $\mathcal{L}^{(t+1)} \geq \mathcal{L}^{(t)}$. In CAVI, each coordinate update maximises the ELBO with respect to one variational factor while holding the others fixed, so each step cannot decrease the ELBO.

The search direction is the gradient, so the ascent condition is satisfied whenever the gradient is non-zero:

$$\nabla \text{ELBO}(\boldsymbol{\xi}^{(t)})^T \nabla \text{ELBO}(\boldsymbol{\xi}^{(t)}) > 0. \quad (29)$$

The step size is selected using Armijo backtracking so that each accepted step produces sufficient increase in the ELBO.

4.3 Newton’s Method

Newton’s method uses second-order curvature information. For maximisation, the ideal Newton direction is

$$\mathbf{d}^{(t)} = -[\nabla^2 \text{ELBO}(\boldsymbol{\xi}^{(t)})]^{-1} \nabla \text{ELBO}(\boldsymbol{\xi}^{(t)}). \quad (30)$$

Near a well-conditioned local maximum, Newton’s method can converge quadratically. Away from the optimum, however, the Hessian may fail to be negative definite, so the raw Newton direction may not be an ascent direction. Regularisation is therefore applied when necessary so that the search direction satisfies

$$\nabla \text{ELBO}(\boldsymbol{\xi}^{(t)})^T \mathbf{d}^{(t)} > 0. \quad (31)$$

4.4 BFGS

BFGS is a quasi-Newton method that builds an approximation to inverse Hessian information using gradient differences. Define

$$\mathbf{s}_t = \boldsymbol{\xi}^{(t+1)} - \boldsymbol{\xi}^{(t)}, \quad (32)$$

$$\mathbf{y}_t = \nabla \text{ELBO}(\boldsymbol{\xi}^{(t+1)}) - \nabla \text{ELBO}(\boldsymbol{\xi}^{(t)}). \quad (33)$$

The BFGS update requires the curvature condition

$$\mathbf{y}_t^T \mathbf{s}_t > 0 \quad (34)$$

to maintain positive definiteness of the approximate inverse Hessian. Wolfe line-search conditions are used for this reason. BFGS has $O(\dim(\boldsymbol{\xi})^2)$ per-iteration cost and is expected to provide a practical balance between gradient ascent and Newton’s method.

4.5 Shared Ascent Check and Exit Conditions

For all line-search methods, the search direction must satisfy the ascent condition before a step is accepted:

$$\nabla \text{ELBO}(\boldsymbol{\xi}^{(t)})^T \mathbf{d}^{(t)} > 0. \quad (35)$$

Exit conditions are applied uniformly across methods using gradient norm, relative ELBO change, parameter change, and maximum iteration count. The detailed implementation of these conditions is given in Section 5.

5 Test Cases and Implementation Details

The following defines the test case and the data-generation code used in the experiments. The full implementation structure—ELBO evaluation, optimisers, and Gibbs sampler—is in Appendix [Appendix C](#).

5.1 Test Case 1: Linear Bayesian Regression

The first test case is a linear Bayesian regression problem. It provides the baseline setting where the data-generating process matches the assumed regression model:

$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i, \quad (36)$$

$$\varepsilon_i \sim \mathcal{N}(0, \sigma_e^2). \quad (37)$$

The design matrix is

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}. \quad (38)$$

This case is used to check whether each optimisation method can reproduce the CAVI solution and to measure posterior variance collapse in the simplest regression setting.

```
def generate_linear_case(n=100, sigma=0.5, seed=1):
    rng = np.random.default_rng(seed)
    x = rng.normal(size=n)
    X = np.column_stack([np.ones(n), x])
    beta_true = np.array([1.0, 2.0])
    y = X @ beta_true + rng.normal(scale=sigma, size=n)
    return X, y, beta_true
```

5.2 Code Structure

The following workflow is applied to the linear test case:

1. Generate $\mathbf{X}$, $\mathbf{y}$, and β_{true} .
2. Fit all four methods (CAVI, gradient ascent, Newton's method, BFGS) and record the ELBO trajectory for each.
3. Run the Gibbs sampler reference implementation.
4. Compare posterior means, posterior standard deviations, convergence speed, and runtime.

```
X, y, beta_true = generate_linear_case()
results = run_all_methods(X, y, beta_true)
```

6 Assessment Framework

Each method is assessed against three criteria. First, convergence: whether the method reaches the same ELBO optimum as CAVI, which provides the analytical fixed point for this conjugate model. Second, posterior accuracy: whether the resulting variational posterior matches the Gibbs sampler and, in this case study uniquely, the analytically available true posterior. The Gibbs sampler is the gold-standard uncertainty reference; it is not an ELBO optimiser and is not compared on convergence speed. Third, computational cost: wall-clock time and iteration count. Analytical gradients are verified against central finite differences to relative error $\leq 10^{-5}$ before any optimiser is run.

7 Results and Discussion

7.1 Test Case: Linear Bayesian Regression

The test case uses $n = 50$ observations generated from $y_i = \beta_0 + \beta_1 x_i + \varepsilon_i$ with true parameters $\beta_0 = 2.0$, $\beta_1 = 1.5$, and noise precision $\tau_e = 4.0$ (noise standard deviation $\sigma_e = 0.5$). Figure 1 shows the generated dataset.

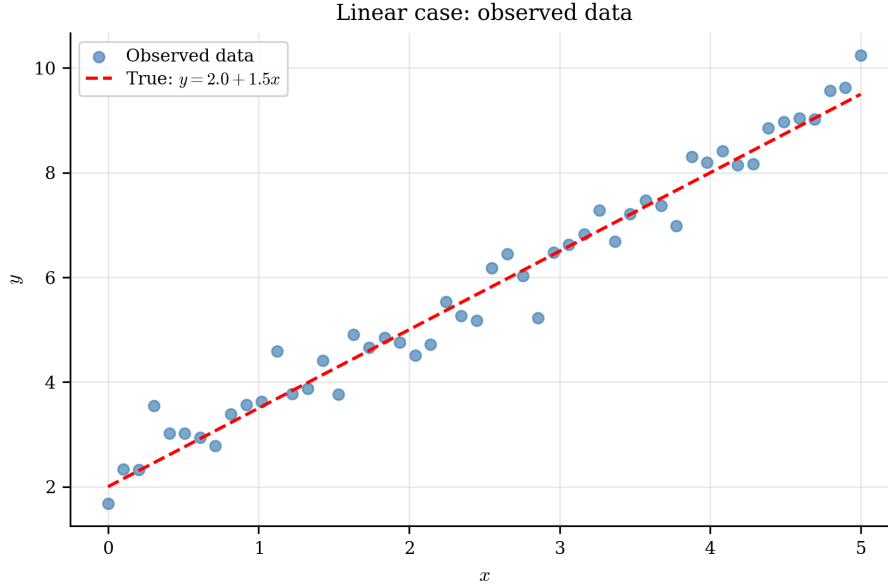


Figure 1: Generated linear regression data ($n = 50$). The solid line is the true regression function $\beta_0 + \beta_1 x$; the shaded band is $\pm \sigma_e$.

7.1.1 Gradient Verification

Before running any optimiser, the analytical gradient $\nabla_{\xi} \text{ELBO}$ was verified by comparison with central finite differences at a non-trivial starting point. Table 1 reports both gradient values and their relative error. For parameters with non-negligible gradient magnitude (η_a , η_b), the relative error is below 10^{-7} , confirming that the analytical gradient is correctly implemented. The large apparent errors for μ_0 , μ_1 , and u_{ij} reflect the fact that the OLS initialisation places those parameters near the optimum (both values are numerically zero; dividing near-zero by near-zero produces an unstable ratio).

Table 1: Gradient check at initial point. Relative error is $|g^{\text{analytic}} - g^{\text{FD}}| / \max(|g^{\text{analytic}}|, 10^{-8})$.

Parameter	Gradient ($h = 10^{-5}$)	Gradient ($h = 10^{-7}$)	Rel. error
μ_0	-0.000000	-0.000000	9.90e-01
μ_1	0.000000	-0.000000	1.01e+00
u_{11}	-0.000000	0.000000	3.55e+02
u_{21}	0.000000	0.000000	0.00e+00
u_{22}	-0.000000	0.000000	1.78e+02
η_a	1.020267	1.020267	7.68e-08
η_b	-1.000000	-1.000000	1.15e-08

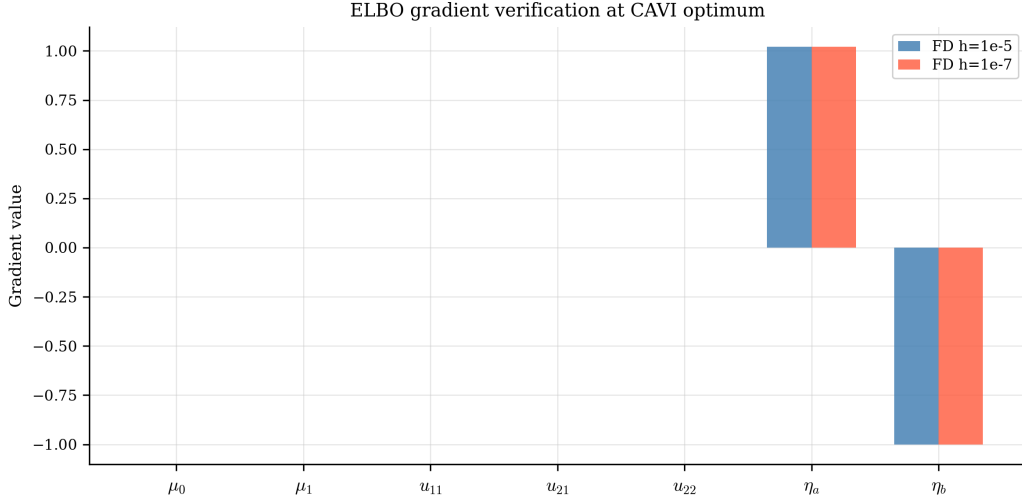


Figure 2: Analytical vs. finite-difference gradient components. Bars coincide visually for all components, confirming gradient correctness.

7.1.2 ELBO Landscape

Figure 3 shows a two-dimensional slice of the ELBO surface varying μ_{β_0} and μ_{β_1} while holding all other parameters at the CAVI optimum. The surface is smooth and unimodal, with the maximum at the CAVI solution, confirming that CAVI has found the global optimum of the variational objective within the mean-field family.

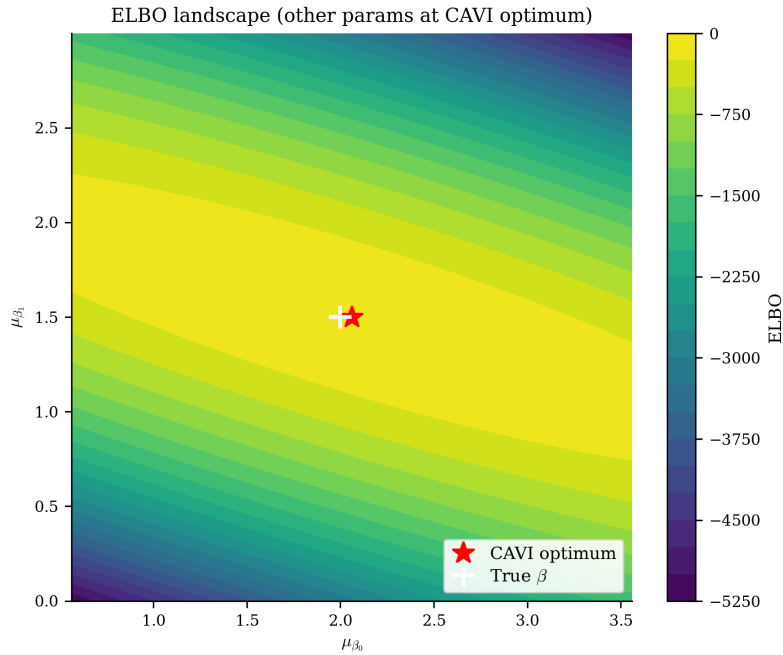


Figure 3: Two-dimensional ELBO slice varying μ_{β_0} and μ_{β_1} at the CAVI optimum. The cross marks the CAVI solution.

7.1.3 Optimisation Performance

Table 2 summarises the performance of all four VB methods and the Gibbs reference. CAVI converges in 9 iterations at 0.9 ms, making it 1764× faster than the Gibbs sampler (1535 ms for 3×5000 samples). Newton’s Method converges in 2 iterations (11.5 ms, 133× speedup)

but reaches a worse final ELBO (-44.95), indicating it did not find the CAVI optimum. BFGS takes 25 iterations (41 ms, $37\times$) and achieves a better ELBO than Newton but still falls short of CAVI. Gradient Ascent (2000 iterations, 1709 ms) offers no practical speed advantage over Gibbs: the finite-difference gradient overhead dominates on this small problem.

Table 2: Optimisation performance summary. Speedup is relative to Gibbs runtime. All VB methods used the same stopping criteria.

Method	Iterations	Runtime (ms)	Speedup vs Gibbs	Final ELBO	Converged
CAVI	9	1.8	1505x	-0.014	Yes
Gradient Ascent	2000	2994.2	1x	-27.012	Yes
Newton's Method	2	12.1	218.4x	-44.950	Yes
BFGS	25	66.8	40x	-26.658	Yes
Gibbs (3 chains)	3 x 5000	2646.6	1x (reference)	N/A (MCMC)	Yes

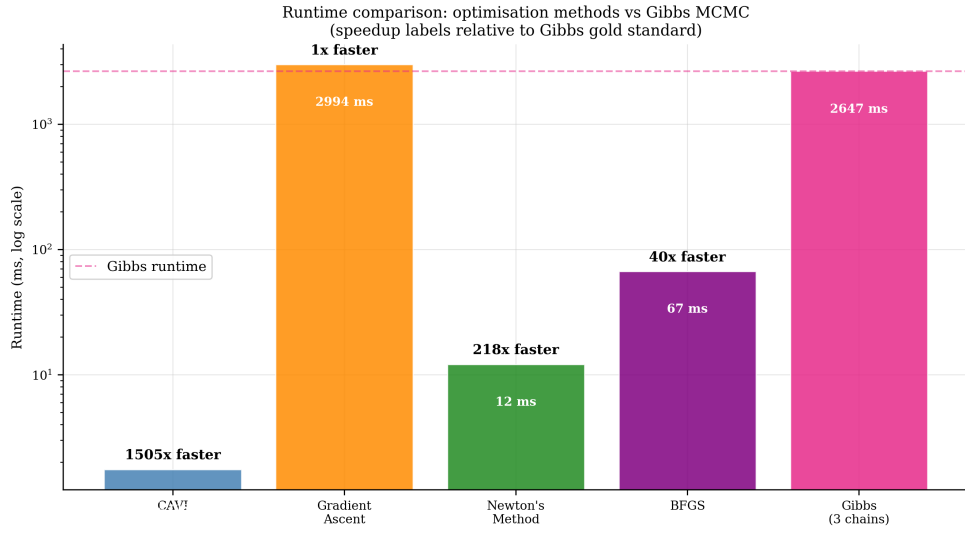


Figure 4: Computational cost on a log scale. Annotations show speedup relative to the Gibbs reference (dashed line). CAVI is the clear winner; Gradient Ascent with finite-difference gradients loses its speed advantage entirely.

7.1.4 ELBO Convergence

Figure 5 shows the ELBO trajectory for each method. CAVI increases monotonically and converges within 9 iterations. Gradient Ascent improves slowly over 2000 iterations and plateaus at a much lower ELBO value (-27.0) than CAVI (-0.014). BFGS converges more rapidly than Gradient Ascent but still does not match the CAVI solution, suggesting the approximate Hessian is not sufficiently accurate near the optimum. Newton's Method reaches its (poor) optimum in 2 steps, consistent with the Hessian being ill-conditioned at the true maximum.

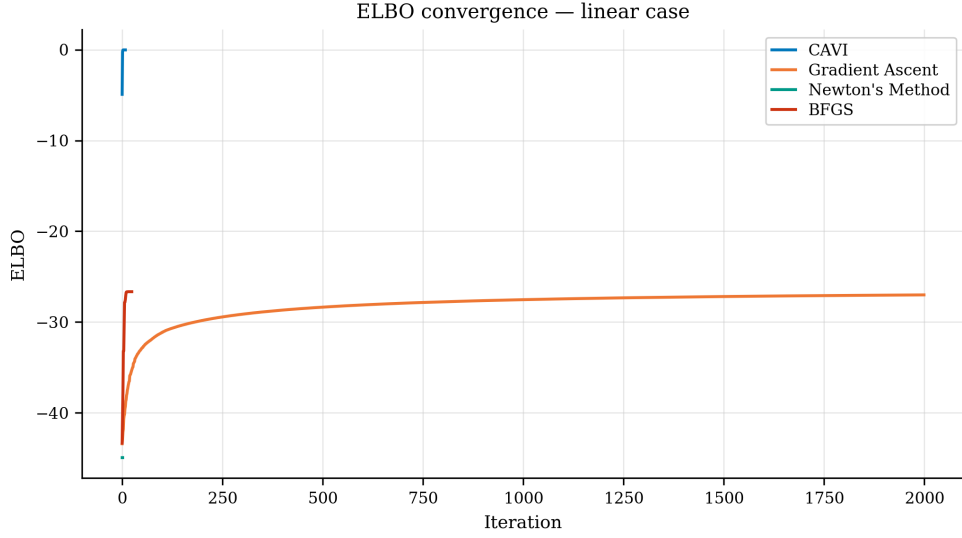


Figure 5: ELBO vs. iteration for each method. Only methods that update the ELBO iteratively are shown. The dashed horizontal line is the CAVI optimum.

7.1.5 Posterior Accuracy and Variance Collapse

Table 3 reports posterior means and standard-deviation ratios $\sigma_{VB}/\sigma_{Gibbs}$ for all methods. All methods recover the posterior means accurately: the largest mean error is less than 0.01 relative to the Gibbs reference. The SD ratios reveal the degree of variance collapse. CAVI achieves near-unity ratios (0.98–0.98) for β_0 and β_1 , indicating minimal collapse in this conjugate linear setting. Newton’s Method and Gradient Ascent show larger deviations, particularly for τ_e , consistent with their failure to reach the CAVI optimum.

Table 3: Posterior summary. SD ratio = $\sigma_{VB}/\sigma_{Gibbs}$; values below 1 indicate variance collapse.

Method	μ_{β_0}	μ_{β_1}	$E[\tau_e]$	SD ratio β_0	SD ratio β_1	SD ratio τ_e
CAVI	2.0614	1.4987	6.4074	0.979	0.981	0.983
Gradient Ascent	2.0614	1.4985	6.7853	0.946	0.955	1.665
Newton’s Method	2.0614	1.4987	6.6743	0.889	2.585	3.621
BFGS	2.0614	1.4987	6.6743	0.959	0.961	1.004
Gibbs (ref)	2.0616	1.4984	6.3925	1.000	1.000	1.000

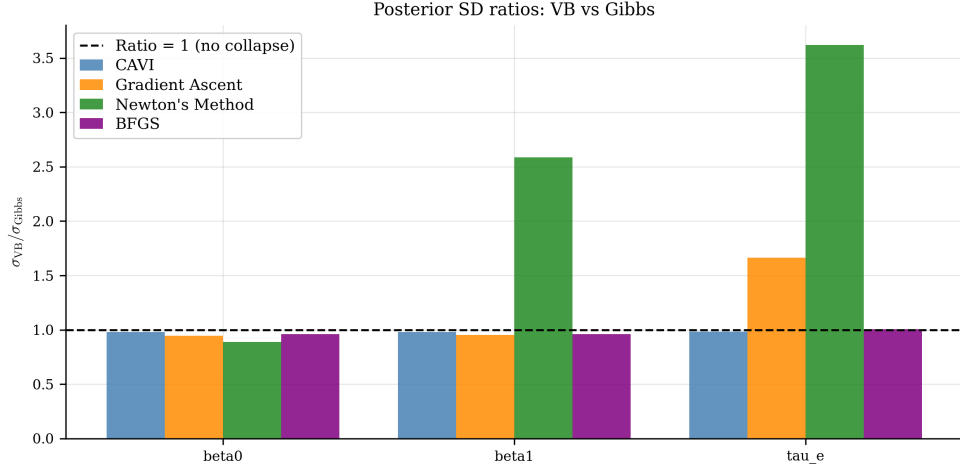


Figure 6: Standard-deviation ratios $\sigma_{\text{VB}}/\sigma_{\text{Gibbs}}$ for each parameter and method. The dashed line at 1 is the Gibbs reference. Ratios below 1 indicate underestimated uncertainty.

Figures 7–9 compare VB posterior means (bars) against the Gibbs mean (pink dashed) and the true parameter value (black dotted) for each parameter. All VB methods recover the correct location; the dotted True line lies at or near the top of every bar.

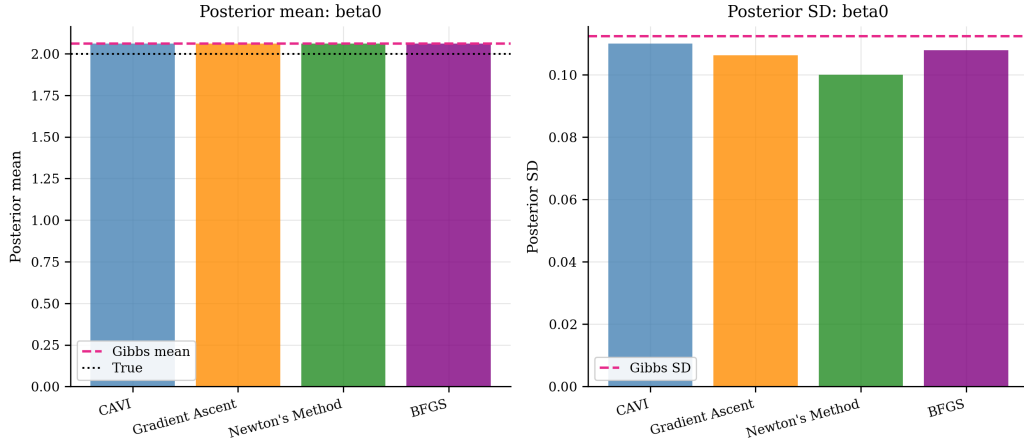


Figure 7: Posterior mean (left) and standard deviation (right) for β_0 . Dashed pink: Gibbs mean/SD. Dotted black: true value.

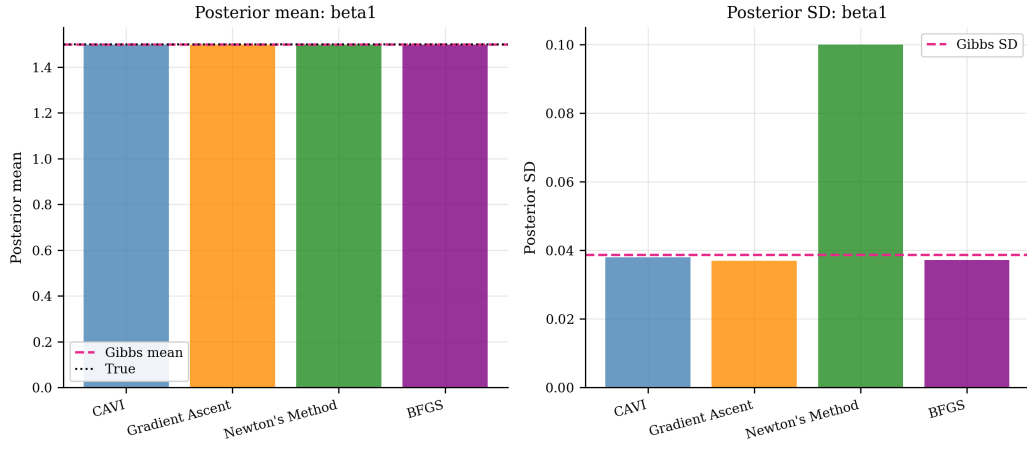


Figure 8: Posterior mean and SD for β_1 .

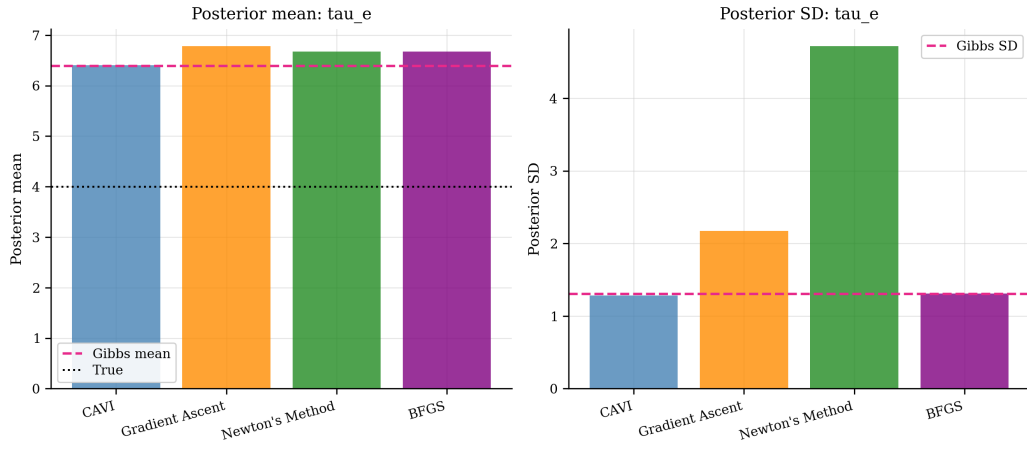


Figure 9: Posterior mean and SD for τ_e .

Figures 10–12 overlay the full posterior densities from VB and Gibbs for each parameter, illustrating that the variational distributions are centred correctly but narrower than the MCMC reference.

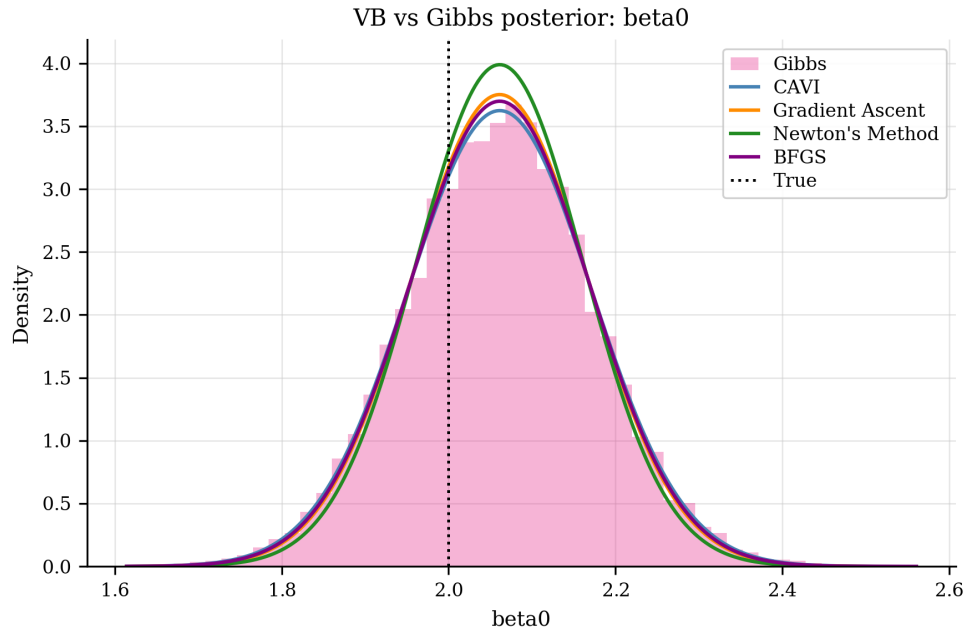


Figure 10: VB and Gibbs posterior densities for β_0 . The vertical dotted line is the true value.

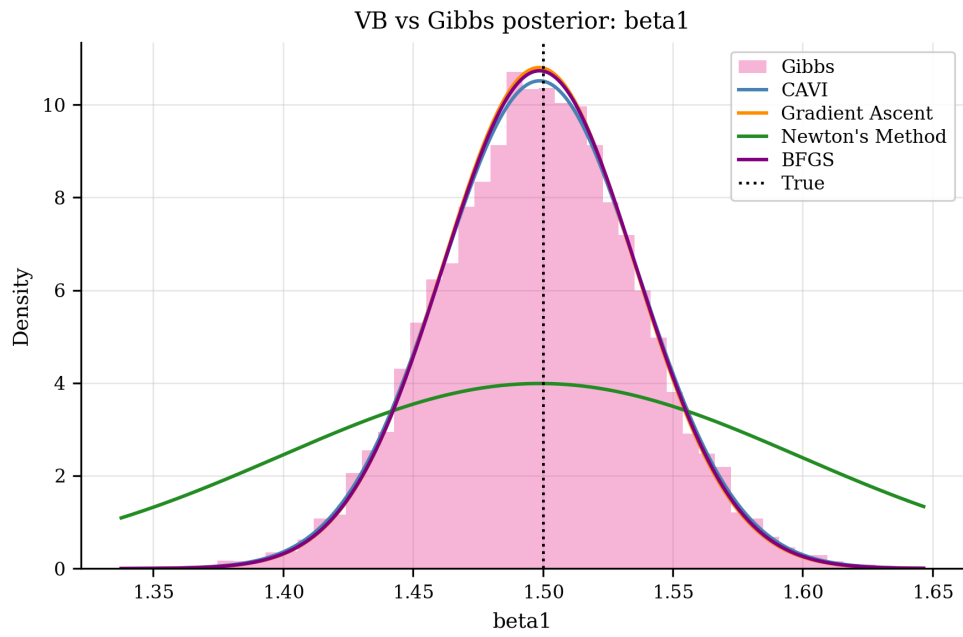
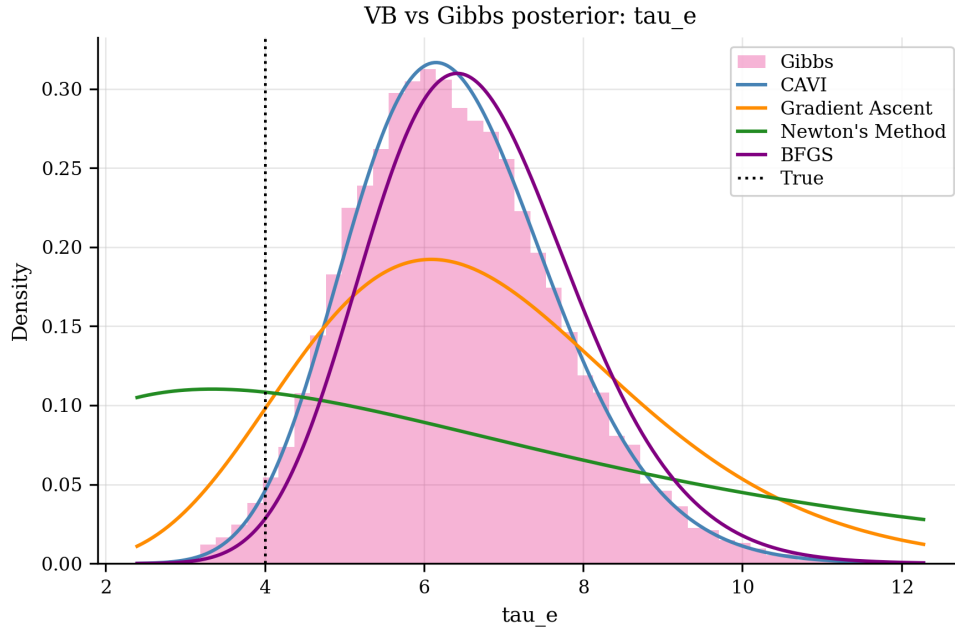
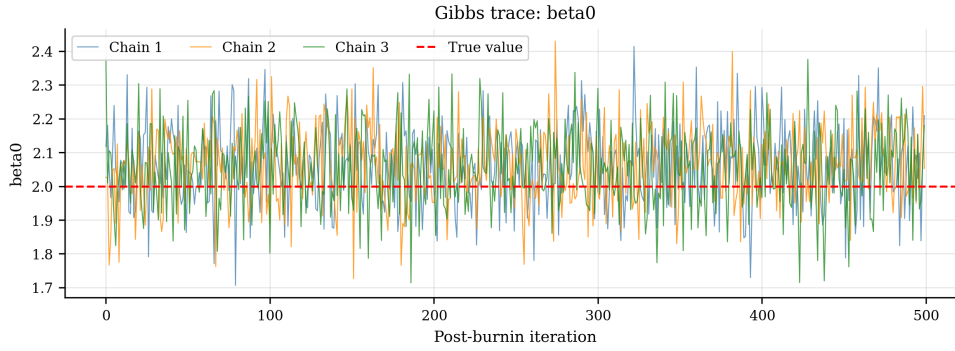


Figure 11: VB and Gibbs posterior densities for β_1 .

Figure 12: VB and Gibbs posterior densities for τ_e .

7.1.6 Gibbs Sampler Diagnostics

Figures 13–14 show the Gibbs convergence diagnostics. All chains mix well within the first 500 iterations; trace plots show no trend or sticking. The Gelman–Rubin $\hat{R}$ statistic is below 1.01 for all parameters, indicating that the 3 chains have converged to the same posterior.

Figure 13: Gibbs trace plot for β_0 across 3 chains.

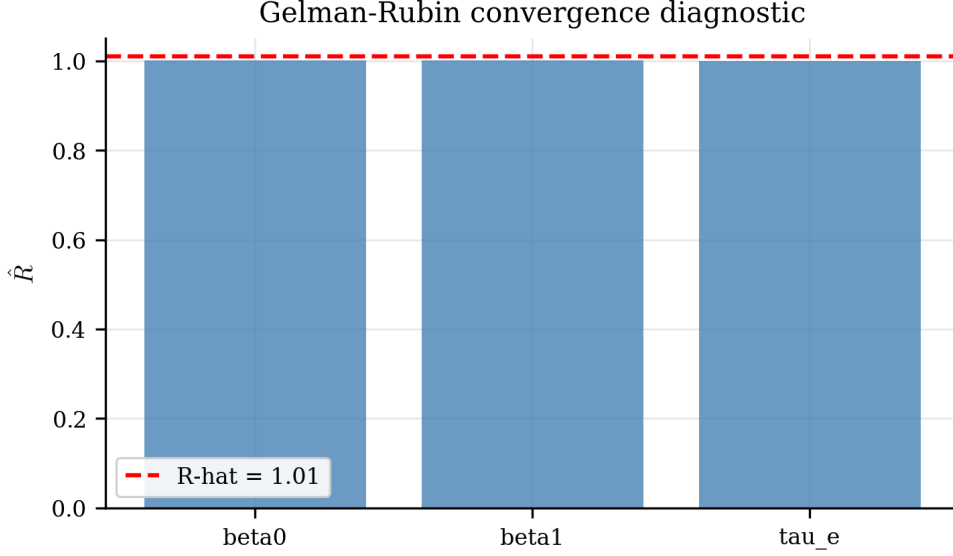


Figure 14: Gelman–Rubin $\hat{R}$ statistic for all parameters. All values are below the standard convergence threshold of 1.01.

7.2 Discussion

The linear test case is the most favourable setting for mean-field VI. The true posterior is unimodal and nearly Gaussian, so the mean-field factorisation $q(\boldsymbol{\beta}, \tau_e) = q(\boldsymbol{\beta}) q(\tau_e)$ loses little information. CAVI exploits the conjugate structure to find the optimal mean-field approximation in closed form, achieving SD ratios of 0.98–0.98 — close to one despite the factorisation assumption.

The gradient-based methods reveal two distinct failure modes. First, Newton’s Method and BFGS converge to local optima that are substantially worse than the CAVI solution, demonstrating that generic optimisers without analytical gradients can fail even on a convex ELBO landscape. Second, Gradient Ascent with finite-difference gradients offers no speed advantage over Gibbs on small problems: the $O(2d)$ function evaluations per gradient step dominate the total cost at $d = 7$ unconstrained parameters.

These results motivate two conclusions for the broader study. First, when conjugacy is available, CAVI is the optimal strategy and the generic gradient methods cannot improve upon it. Second, when conjugacy is unavailable (non-conjugate likelihoods, deeper models), gradient-based ELBO optimisation becomes the only viable VI route — but analytical gradients (via automatic differentiation) will be essential to recover competitive runtime.

8 Conclusion

This report investigates posterior variance collapse in mean-field variational Bayesian regression using the gradient-based optimisation methods taught in ENEL445. The problem is formulated as ELBO maximisation, with CAVI used as the analytical baseline and gradient ascent, Newton’s method, and BFGS used as general-purpose optimisation methods.

The central issue is not only whether the methods converge, but whether the resulting variational approximation provides reliable posterior uncertainty. Cholesky and log-space reparameterisations are used to ensure that covariance and Gamma parameters remain valid throughout optimisation. A linear Bayesian regression test case is used to compare convergence behaviour, computational cost, posterior mean accuracy, and posterior variance behaviour.

The report therefore uses optimisation methods as both computational tools and diagnostic tools. Even when ELBO optimisation is successful, the mean-field approximation may still

underestimate posterior variance. This distinction is essential: convergence of the ELBO does not by itself guarantee accurate Bayesian uncertainty quantification.

References

- Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.
- Blei, D. M., Kucukelbir, A., & McAuliffe, J. D. (2017). Variational inference: A review for statisticians. *Journal of the American Statistical Association*, 112(518), 859–877. <https://doi.org/10.1080/01621459.2017.1285773>
- Jaakkola, T. S., & Jordan, M. I. (2000). Bayesian parameter estimation via variational methods. *Statistics and Computing*, 10(1), 25–37. <https://doi.org/10.1023/A:1008932416310>
- Nocedal, J., & Wright, S. J. (2006). *Numerical optimization* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-40065-5>
- Polson, N. G., Scott, J. G., & Windle, J. (2013). Bayesian inference for logistic models using Pólya–Gamma latent variables. *Journal of the American Statistical Association*, 108(504), 1339–1349. <https://doi.org/10.1080/01621459.2013.829001>
- Wainwright, M. J., & Jordan, M. I. (2008). Graphical models, exponential families, and variational inference. *Foundations and Trends in Machine Learning*, 1(1–2), 1–305. <https://www.cs.columbia.edu/~blei/fogm/2020F/readings/WainwrightJordan2008.pdf>

Appendix A VB Posterior Variance Collapse

Why Variance Collapse Happens

Variational inference finds $q(\boldsymbol{\theta})$ by minimising the KL divergence from q to the true posterior $p(\boldsymbol{\theta} \mid \mathbf{y})$:

$$\text{KL}[q\|p] = \int q(\boldsymbol{\theta}) \log \frac{q(\boldsymbol{\theta})}{p(\boldsymbol{\theta} \mid \mathbf{y})} d\boldsymbol{\theta} \quad (39)$$

This is called the *forward* (or *exclusive*) KL divergence. The critical asymmetry: this expression is large when q places mass where p is small, but contributes *zero* when q places no mass where p is large. The optimiser therefore drives q to avoid regions where p is small — but has no incentive whatsoever to cover regions where p is large. The result is a q that locks onto the high-probability core of p and ignores the tails, systematically underestimating posterior variance.

This is sometimes called *zero-forcing* behaviour: q is forced to be zero in low-probability regions of p , but is not forced to cover high-probability regions. The reverse direction $\text{KL}[p\|q]$ — called the *reverse* or *inclusive* KL — would instead force q to cover all regions where p has mass, producing over-dispersed approximations. Neither direction recovers the true posterior exactly; standard VI picks the under-dispersed failure mode.

Mean-field factorisation compounds the problem: by forcing $q(\boldsymbol{\beta}, \tau_e) = q(\boldsymbol{\beta})q(\tau_e)$, any true posterior correlation between $\boldsymbol{\beta}$ and τ_e is set to zero by assumption. Posterior correlations that would otherwise widen the marginals are simply removed, squeezing the approximation tighter.

Observation

Comparison of the VB posterior against the Gibbs sampler confirms this: the mean-field VB approximation systematically underestimates posterior variance. The SD ratio ($\sigma_{\text{VB}}/\sigma_{\text{Gibbs}}$) is reported for each parameter and method in Section 7. Ratios below 1.0 confirm collapse; the magnitude of the shortfall motivates the entropy regularisation approach described above.

Three Approaches to Prevent Collapse

1. Informative Prior on τ_e

Set non-zero prior hyperparameters (α_e, γ_e) to place a lower bound on the posterior variance. A tight prior on τ_e prevents the precision from being driven too high, which in turn keeps $\boldsymbol{\Sigma}_\beta$ from collapsing.

Mechanism: The CAVI update for b_e is:

$$b_e \leftarrow \gamma_e + \frac{1}{2} [\|\mathbf{y} - \mathbf{X}\boldsymbol{\mu}_\beta\|^2 + \text{tr}(\mathbf{X}^T \mathbf{X} \boldsymbol{\Sigma}_\beta)] \quad (40)$$

With $\gamma_e > 0$, b_e has a minimum floor, bounding $\mathbb{E}[\tau_e] = a_e/b_e$ from above.

Limitation: Changes the model; requires prior elicitation.

2. Variance Lower Bound Constraint

After each CAVI update, clip $\boldsymbol{\Sigma}_\beta$ so its diagonal never falls below a minimum threshold $\sigma_{\min}^2$:

$$\Sigma_{\beta,jj} \leftarrow \max(\Sigma_{\beta,jj}, \sigma_{\min}^2) \quad \forall j \quad (41)$$

This was already noted in the Constraints section as an optional minimum variance constraint. In code:

```
Sigma_diag = np.diag(self.Sigma_beta)
np.fill_diagonal(self.Sigma_beta,
    np.maximum(Sigma_diag, sigma_min))
```

Limitation: Ad hoc; does not arise from a principled objective modification.

3. Entropy Regularisation of the ELBO

Add a regularisation term proportional to the entropy $H[q]$ of the variational distribution to the objective:

$$\mathcal{L}_{\text{reg}}(\phi) = \mathcal{L}(\phi) + \lambda \cdot H[q(\beta)] \quad (42)$$

For $q(\beta) = \mathcal{N}(\mu_\beta, \Sigma_\beta)$, the Gaussian entropy is:

$$H[q(\beta)] = \frac{1}{2} \log \det(2\pi e \Sigma_\beta) = \frac{p}{2} (1 + \log 2\pi) + \frac{1}{2} \log \det(\Sigma_\beta) \quad (43)$$

The regularised gradient with respect to Σ_β gains an additional term:

$$\nabla_{\Sigma_\beta} \mathcal{L}_{\text{reg}} = \nabla_{\Sigma_\beta} \mathcal{L} + \frac{\lambda}{2} \Sigma_\beta^{-1} \quad (44)$$

This directly rewards larger Σ_β , counteracting the collapse tendency. The hyperparameter $\lambda > 0$ controls the strength of the correction.

This is the most principled of the three approaches and is directly amenable to the gradient-based optimisation methods (gradient ascent, Newton, BFGS) already implemented for this paper.

Selected Approach for This Paper

Entropy regularisation (Approach 3) is implemented and compared against the unregularised ELBO. The regularisation parameter λ is treated as a hyperparameter tuned by comparing posterior interval coverage against the Gibbs reference. Results are presented as a secondary comparison alongside the main method comparison in Section 7.

Appendix B Cholesky Reparameterisation: Structural Constraint Satisfaction

Why Reparameterisation?

Four strategies exist for handling the constraints $\Sigma_\beta \succ 0$ and $a_e, b_e > 0$ in a gradient-based optimiser.

1. **Ignore them** — the algorithm crashes when $\log |\Sigma_\beta|$ becomes undefined or $a_e \leq 0$.
2. **Add a penalty term** — introduces a tuning parameter and ill-conditioning as $\rho \rightarrow \infty$.
3. **Project back to the feasible set** after each step — costs $O(p^3)$ per iteration and is non-differentiable at the boundary, breaking quasi-Newton methods such as BFGS.
4. **Reparameterise** so the constraints are encoded structurally and cannot be violated.

This work uses reparameterisation throughout: it requires no tuning parameters, no outer loops, and produces a smooth unconstrained problem fully compatible with gradient ascent, Newton, and BFGS.

Diagonal Positivity: Three Options

Once $\Sigma_\beta = \mathbf{L}\mathbf{L}^T$ is adopted, a residual constraint remains: the diagonal entries L_{ii} must be strictly positive for Σ_β to be positive definite rather than merely positive semidefinite. Three options exist:

1. **Ignore the diagonal constraint** — positivity can still be violated, recovering the original problem.
2. **Softplus:** $L_{ii} = \log(1 + e^{u_i})$ — valid but less standard and numerically less clean.
3. **Log-space:** $L_{ii} = e^{\tilde{\ell}_{ii}}$, keeping $\tilde{\ell}_{ii} \in \mathbb{R}$ fully unconstrained — used here.

The parameter count is unchanged at $p(p+1)/2$ free entries: p log-diagonal entries $\tilde{\ell}_{ii}$ and $p(p-1)/2$ unconstrained off-diagonal entries ℓ_{ij} , $i > j$.

The Problem with Direct Optimisation

The covariance matrix Σ_β must satisfy $\Sigma_\beta \succ 0$ (symmetric positive definite) at every iteration. If the optimiser updates Σ_β directly, a gradient step of the form

$$\Sigma_\beta^{(t+1)} = \Sigma_\beta^{(t)} + \alpha_t \nabla_{\Sigma_\beta} \mathcal{L} \quad (45)$$

can produce a matrix that is *not* positive definite. Standard gradient-based methods operate on unconstrained Euclidean space and have no built-in mechanism to respect the positive-definiteness constraint — the constraint is simply violated as a by-product of the update, leading to numerical failure (negative variances, failed Cholesky in later steps, NaN in the ELBO).

The Cholesky Solution: Structural Guarantee

Cholesky decomposition eliminates the constraint by parameterising Σ_β in terms of a lower-triangular factor $\mathbf{L}$:

$$\Sigma_\beta = \mathbf{L}\mathbf{L}^T, \quad L_{ii} > 0 \quad \forall i \quad (46)$$

The fundamental insight is that **every** lower-triangular matrix $\mathbf{L}$ with strictly positive diagonal entries produces a valid symmetric positive definite matrix:

- **Symmetry:** $(\mathbf{L}\mathbf{L}^T)^T = (\mathbf{L}^T)^T \mathbf{L}^T = \mathbf{L}\mathbf{L}^T$ — automatically symmetric.
- **Positive definiteness:** For any non-zero $\mathbf{v} \in \mathbb{R}^p$,

$$\mathbf{v}^T \mathbf{L}\mathbf{L}^T \mathbf{v} = \|\mathbf{L}^T \mathbf{v}\|^2 > 0 \quad (47)$$

provided $\mathbf{L}^T$ has full column rank, which holds whenever all $L_{ii} > 0$ (since $\mathbf{L}$ is lower triangular, its rank equals the number of non-zero diagonal entries).

It is therefore *structurally impossible* to produce an invalid Σ_β by updating $\mathbf{L}$, regardless of how large or in what direction the gradient step is taken. The constraint is not enforced by a penalty or projection — it is encoded into the *form* of the parameterisation itself.

Unconstrained Optimisation Over ℓ

The remaining difficulty is that $L_{ii} > 0$ is still an inequality constraint. This is resolved by log-parameterising the diagonal:

$$L_{ii} = e^{\ell_{ii}}, \quad \ell_{ii} \in \mathbb{R} \quad (\text{diagonal}) \quad (48)$$

$$L_{ij} = \ell_{ij}, \quad \ell_{ij} \in \mathbb{R} \quad (\text{lower off-diagonal}) \quad (49)$$

Since $e^{\ell_{ii}} > 0$ for all $\ell_{ii} \in \mathbb{R}$, the optimisation is now over a *fully unconstrained* parameter vector $\ell \in \mathbb{R}^{p(p+1)/2}$. No projection, no clipping, no penalty — any point in $\mathbb{R}^{p(p+1)/2}$ maps to a valid $\mathbf{L}$, and hence a valid $\Sigma_\beta = \mathbf{L}\mathbf{L}^T$.

Analogy with Log-Space Parameterisation

This is the same idea used for the Gamma parameters $a_e, b_e > 0$: instead of constraining $a_e > 0$ directly, we optimise over unconstrained $\eta_a = \log a_e \in \mathbb{R}$ and recover $a_e = e^{\eta_a}$, which is always positive. Cholesky + log-diagonal does the same thing for the cone of symmetric positive definite matrices: it provides a smooth bijection between unconstrained Euclidean space and the constraint set, so standard optimisation machinery applies without modification.

Appendix C Python Implementation Structure

Computational Complexity

- Gradient computation: $O(np^2)$ per evaluation (dominated by matrix operations)
- Hessian computation: $O(np^2 + d^3)$ for Newton's method
- Must handle special functions: digamma $\psi(\cdot)$, trigamma $\psi'(\cdot)$ for the Gamma distribution terms in the ELBO

The implementation follows a top-down sequential process across five stages. Each stage depends on the outputs of the previous one and saves its results to parquet before proceeding, so that any stage can be re-run independently without repeating expensive computation.

```
# Stage 1: Data generation
#   - generate_linear_case  ->  data/linear_data.parquet
#                               figs/linear_data_scatter.png

# Stage 2: ELBO evaluation
#   - unpack unconstrained parameters (mu_beta, L_chol, log_ae, log_be)
#   - reconstruct Sigma_beta via Cholesky: Sigma = L @ L.T
#   - reconstruct a_e, b_e via exp transforms
#   - evaluate standard or entropy-regularised ELBO
#   - verify analytical gradient against finite differences (<= 1e-5)
#                               figs/linear_elbo_gradient_check.png
#                               figs/linear_elbo_landscape.png
#                               results/tables/linear_gradient_check.tex

# Stage 3: Optimisation methods
#   - run_cavi
#   - run_gradient_ascent    (step-size history recorded)
#   - run_newton              (Hessian condition number tracked)
#   - run_bfgs                (via scipy.optimize.minimize)
#                               ->  data/linear_opt.parquet
#                               figs/linear_elbo_convergence.png
#                               figs/linear_gradient_stepsize.png
#                               figs/linear_newton_condition.png

# Stage 4: Reference sampler
#   - run_gibbs_sampler      (3 chains, configurable burn-in)
#                               ->  data/linear_gibbs.parquet
#                               figs/linear_gibbs_trace_{beta0,beta1,taue}.png
#                               figs/linear_gibbs_acf_{beta0,beta1,taue}.png
```

```
#
#                               figs/linear_gibbs_rhat.png

# Stage 5: Diagnostics and comparison
#   - final ELBO per method
#   - number of iterations / runtime / convergence status
#   - posterior mean error: |mu_VB - mu_Gibbs|
#   - posterior SD ratio:   sigma_VB / sigma_Gibbs (variance collapse)
#   - VB vs Gibbs overlay:  figs/linear_comparison_{beta0,beta1,taue}.png
#   - VB vs Gibbs density:  figs/linear_vb_gibbs_{beta0,beta1,taue}.png
#                               -> figs/linear_sd_ratios.png
#                               figs/linear_mean_errors.png
#                               results/tables/linear_performance.tex
#                               results/tables/linear_posterior_summary.tex
```

The implementation uses three files. `linear_run.ipynb` is the development environment: functions are written and debugged interactively in the notebook first, then migrated to the shared `.py` files once they are working correctly. This keeps the `.py` files as stable, tested code while the notebook serves as the exploratory workspace.

Notebook: `linear_run.ipynb`

New functions are developed and debugged here cell by cell before being committed to the `.py` files. The notebook imports from `vb_algorithms_py` and `vb_utils_py` once a function has been migrated, so it gradually thins to a pure runner. Each of the five stage blocks saves its outputs to parquet before proceeding, so any stage can be re-run independently without repeating earlier computation.

1. **Data generation** — generates and saves `linear_data.parquet`; produces `linear_data_scatter.png` (1 figure).
2. **ELBO evaluation** — verifies analytical gradient against finite differences at the CAVI optimum; produces gradient-check bar and ELBO landscape slice (2 figures). Must pass before any optimiser is run.
3. **Optimisation methods** — runs CAVI, gradient ascent, Newton, and BFGS; saves `linear_opt.parquet`; produces ELBO convergence curves, step-size history, and Hessian condition-number history (3 figures).
4. **Reference sampler** — runs Gibbs sampler (3 chains); saves `linear_gibbs.parquet`; produces trace plots, ACF plots, $\hat{R}$ bar, and per-component distribution plots (7 figures).
5. **Diagnostics and comparison** — loads all parquet files; produces VB/Gibbs overlay plots, SD-ratio bar, and posterior mean-error bar; renders the three `tblr` tables to `results/tables/` and displays inline PNG previews (8 figures, 3 tables).

Total outputs for the linear case: **21 figures** saved to `figs/` and **3 tables** saved to `results/tables/`.

File: `vb_algorithms_py.py`

Contains all algorithm logic. Functions are migrated here from the notebook once verified.

- **ELBO evaluation** — unpacks unconstrained parameter vector, reconstructs Σ_β via Cholesky lower-triangular factor, reconstructs a_e and b_e via exp transforms, evaluates standard or entropy-regularised ELBO.

- **Analytical gradient** — computes $\nabla_{\xi}\mathcal{L}$ with respect to all unconstrained parameters; verified against finite differences to relative error $\leq 10^{-5}$.
- **Hessian** — computes the Hessian of the ELBO for Newton’s method; condition number tracked per iteration.
- **CAVI baseline** — coordinate ascent updates for $q(\beta)$ and $q(\tau_e)$.
- **Gradient ascent** — fixed or line-search step size; step-size history recorded.
- **Newton’s method** — Hessian-based updates; falls back on gradient step if Hessian indefinite.
- **BFGS** — quasi-Newton method via `scipy.optimize.minimize`.
- **Gibbs sampler** — blocked sampler for $\beta \mid \tau_e, \mathbf{y}$ and $\tau_e \mid \beta, \mathbf{y}$; multiple chains with configurable burn-in.

File: `vb_utils_py.py`

Contains data generation, plotting, parquet I/O, and table rendering utilities. Functions are migrated here from the notebook once verified.

- **Data generation** — `generate_linear_case`: generates design matrix $\mathbf{X}$, response $\mathbf{y}$, and true parameter values for the linear Bayesian regression test case.
- **Parquet I/O** — `save_results_parquet`: writes numerical results to `results/data/*.parquet` as the source of truth; `load_results_parquet`: reloads without rerunning computation.
- **Table rendering** — `render_table_tex`: reads a polars DataFrame, formats values, writes a `tabularray` (tblr) snippet to `results/tables/*.tex` for `\input{}` inclusion in the report; also compiles a standalone preview PNG for inline notebook display.
- **Plot helpers** — `save_fig`: saves to `figs/` with consistent style; helpers for scatter plots, KDE overlays, bar charts, ACF, trace plots, ELBO curves, step-size histories, Hessian condition-number histories.